

Same model, same math, eleven times faster

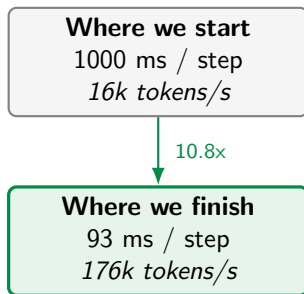
We have GPT-2 124M written, initialized and training. One step takes **1000 milliseconds** on an A100.

By the end of this lecture the same step takes **93 milliseconds**.

Five changes get us there, each one or two lines of code. And here is the part worth sitting with:

none of them change what the network computes.

We are not building a better model today. We are learning to **stop wasting the machine we already rented**.



identical model, identical data

Outline

Know your hardware

Rungs 1 and 2: spend fewer bits

Rung 3: stop making round trips

Rungs 4 and 5: what the compiler cannot find

The ladder, and what it means for you

Know your hardware

Before you optimize anything, ask what the machine can actually do.

Three questions, always in this order

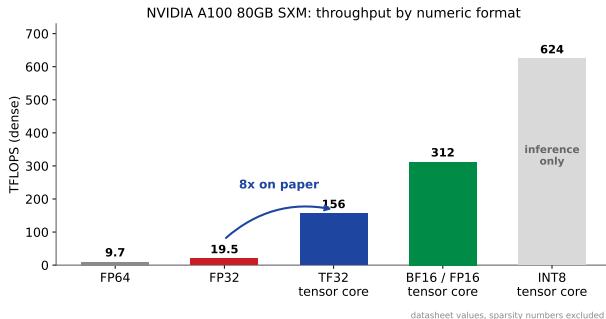
1. **What hardware do I have?** (nvidia-smi)
2. **What does it offer?** (the datasheet)
3. **Am I actually getting it?** (measure, do not assume)

For this lecture: one **NVIDIA A100 80GB SXM**, rented by the hour.

Every optimization below comes from taking question 3 seriously. The datasheet will promise us 8x, and we will not get it. **Understanding that gap is the lecture.**

Optimization without measurement is decoration. Every number in this deck was measured on the same box, one change at a time.

What the machine offers, by numeric format



The same silicon delivers wildly different throughput depending on how many bits you spend per number. TFLOPS = trillions of **floating-point operations (FLOPs)** per second. Note the shape: **fewer bits, more throughput.**

```

train_gpt2.py M X
train_gpt2.py > ...
220 device = cuda
221 elif hasattr(torch.backends, "mps") and torch.backends.mps.is_available():
222     device = "mps"
223     print(f"using device: {device}")
224 torch.manual_seed(1337)
225 if torch.cuda.is_available():
226     torch.cuda.manual_seed(1337)
227
228 train_loader = DataLoaderLite(B=4, T=32)
229
230 # get logits
231 model = GPT(GPTConfig())
232 model.to(device)
233
234 # optimize!
235 optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)
236 for i in range(50):
237     x, y = train_loader.next_batch()
238     x, y = x.to(device), y.to(device)
239     optimizer.zero_grad()
240     logits, loss = model(x, y)
241     import code; code.interact(local=locals())
242     loss.backward()
243     optimizer.step()
244     print(f"step {i}, loss: {loss.item()}")
245
246 import sys; sys.exit(0)
247
248 # prefix tokens
249 model.eval()
250
251 (InteractiveConsole)
>>> logits.dtype
torch.float32
>>> exit(9)
File "c:\console", line 1
exit(9)
^
SyntaxError: unmatched ')'

```

The Most Powerful Compute Platform for Every Workload

The NVIDIA A100 Tensor Core GPU delivers unprecedented acceleration—at every scale—to power the world's highest-performing elastic data centers for AI, data analytics, and high-performance computing (HPC) applications. As the engine of the NVIDIA data center platform, A100 provides up to 20X higher performance over the prior NVIDIA Volta™ generation. A100 can efficiently scale up or be partitioned into seven isolated GPU instances with Multi-Instance GPU (MIG), providing a unified platform that enables elastic data centers to dynamically adjust to shifting workload demands.

NVIDIA A100 Tensor Core technology supports a broad range of math precisions, providing a single accelerator for every workload. The latest generation A100 80GB doubles GPU memory and debuts the world's fastest memory bandwidth at 2 terabytes per second (TB/s), speeding time to solution for the largest models and most massive datasets.

A100 is part of the complete NVIDIA data center solution that incorporates building blocks across hardware, networking, software, libraries, and optimized AI models and applications from the NVIDIA NGC™ catalog. Representing the most powerful end-to-end AI and HPC platform for data centers, it allows researchers to deliver real-world results and deploy solutions into production at scale.

NVIDIA A100 TENSOR CORE GPU SPECIFICATIONS (SXM4 AND PCIE FORM FACTORS)

	A100 80GB PCIe	A100 80GB SXM
FP64	9.7 TFLOPS	
FP64 Tensor Core	19.5 TFLOPS	
FP32	19.5 TFLOPS	
Tensor Float 32 (TF32)	156 TFLOPS 312 TFLOPS*	
BFLOAT16 Tensor Core	312 TFLOPS 624 TFLOPS*	
FP16 Tensor Core	312 TFLOPS 624 TFLOPS*	
INT8 Tensor Core	624 TOPS 1248 TOPS*	
GPU Memory	80GB HBM2e	80GB HBM2e
GPU Memory Bandwidth	1,935GB/s	2,039GB/s
Max Thermal Design Power (TDP)	300W	400W***
Multi-Instance GPU	Up to 7 MIGs @ 10GB	Up to 7 MIGs @ 10GB
Form Factor	PCIe	SXM

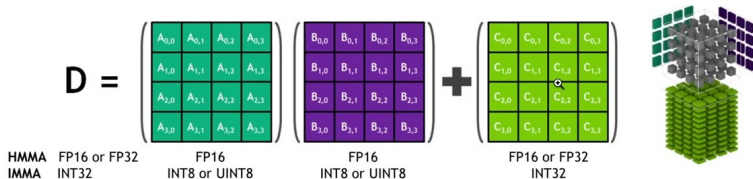
Interconnect

Server Options



What a tensor core actually is

Not a mysterious accelerator. A single **instruction** that multiplies two small matrices and adds a third, in one shot.



Source: NVIDIA developer blog, shown in Karpathy (2024), 1:28:45

Every large matrix multiply in your model gets chopped into thousands of these fixed-size operations. Those rows on the previous chart are simply **which precisions this one instruction will accept** - which is why lower precision buys throughput at all. (HMMA and IMMA are just the float and integer names for the instruction.)

Why matrix multiplication is the whole game

A Transformer is, computationally, **a stack of matrix multiplies** with cheap things sandwiched between them: residual adds, layer norms, GELU.

And at the 124M scale, one matmul dwarfs all the others:

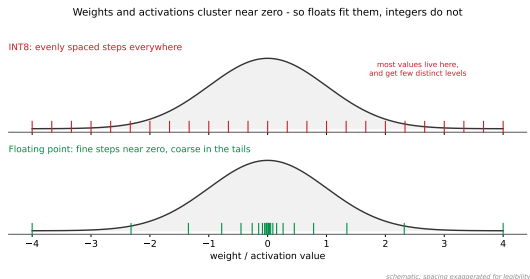
Matrix multiply	Shape	Arithmetic
one MLP (feed-forward) projection	768 $\rightarrow$ 3072	0.077 TFLOP
the classifier	768 $\rightarrow$ 50257	1.26 TFLOP

Projecting to the vocabulary is about **16x** the arithmetic of a single MLP projection. The vocabulary is large and the embedding is small, so that one layer dominates the whole network.

Hold on to this table. In rung 5 we speed the model up by making **this exact matmul bigger**.

Why INT8 sits out this lecture

INT8 is the fastest row on that chart, and we will not use it for **training**.



Integers space their levels **evenly**, so they spend most of their precision where almost no values live. Floats put fine steps near zero, which is where the values actually are.

INT8 is a standard **inference** format. For training we stay with floating point and only argue about *how many* bits.

The bottleneck is not the arithmetic

Here is the fact that explains four of our five speedups.

Tensor cores can multiply far faster than memory can **deliver numbers to them**. Most of the time they are idle, waiting.

If you hit 60% hardware utilization, you are doing extremely well.

Half the time, in a well-tuned job, your tensor cores are doing nothing because the data has not arrived.

This is why lower precision pays **twice**: the multiply gets faster, *and* every number becomes smaller to move and store. The second effect is usually the bigger one.

Measure it properly, or you measure nothing

Your CPU does not run the network. It **queues work** for the GPU and moves on.

So a naive timer stops the clock when the *queueing* finishes, not when the *computing* does.

```
t0 = time.time()
# ... forward, backward, optimizer step ...
torch.cuda.synchronize() # wait for the GPU to actually finish
dt = time.time() - t0
```

Without `synchronize()` you will happily report a speedup that does not exist. Also: **ignore the first iteration** - it pays for memory allocation and other one-time setup, so it is never representative.

Baseline, batch 16×1024 tokens: **1000 ms/step**, about **16k tokens/s**.

Batch size, and the metric that survives it

- ▶ **Max out the batch that fits.** If you hit an out-of-memory error, halve it: 16, then 8, then 4. If you have headroom, go up.
- ▶ **Keep it rich in factors of two.** 8, 16, 32 and 48 all divide by 16. 17 divides by nothing, and rung 5 explains why that hurts.
- ▶ **Report tokens per second, not milliseconds per step.** The moment the batch size changes, milliseconds stop being comparable between runs. Tokens per second is the number you actually care about.

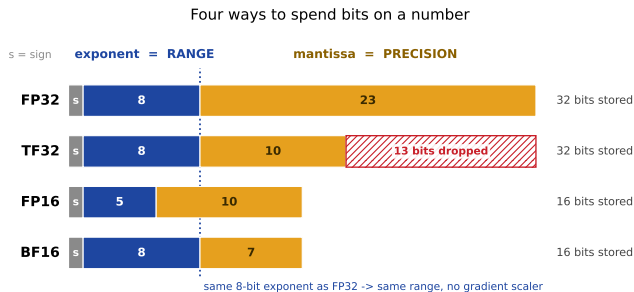
$16 \times 1024 = 16,384$ tokens per step, at 1000 ms $\rightarrow$ **16,384 tokens/s.**

Careful: batch size is a **throughput** knob *and* an optimization knob. Changing it changes your training run, not just its speed. Decoupling the two is what gradient accumulation is for - the next deck.

Spend fewer bits

The cheapest speedup available is to stop moving 32 bits when 16 will do.

Anatomy of a float: range and precision are separate budgets



The **exponent** decides how big and how small a number you can represent (the *range*). The **mantissa** decides how finely you can tell nearby numbers apart (the *precision*). Every format below is a different opinion about how to split the budget.

Rung 1: TensorFloat-32 (TF32)

TF32 crops the mantissa from 23 bits to 10 - but only **inside the tensor-core instruction**.

- ▶ Your inputs are FP32 (32-bit floating point). Your outputs are FP32.
- ▶ The internal accumulation is FP32.
- ▶ Nothing in your code changes dtype. **You cannot see it from Python.**

```
torch.set_float32_matmul_precision('high')
```

This is why TF32 is such a sweet spot: one line, no code changes anywhere else, slightly more approximate results that you will not notice in training. It only exists on Ampere and newer.

Predict first: the datasheet promised 8x

FP32 gives 19.5 TFLOPS. TF32 gives 156 TFLOPS. That is exactly **8x**.

We flip the switch and re-time the step. What do we get?

Predict first: the datasheet promised 8x

FP32 gives 19.5 TFLOPS. TF32 gives 156 TFLOPS. That is exactly **8x**.

We flip the switch and re-time the step. What do we get?

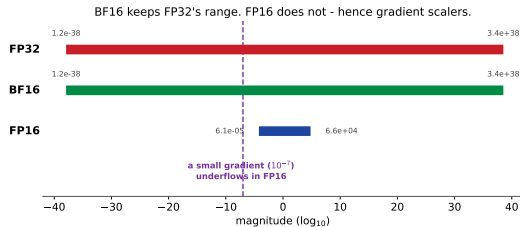
1000 ms → 333 ms. Three times, not eight.

We made the *multiply* eight times faster and left the *traffic* completely unchanged. Every number in and out of that instruction is still a full 32 bits moving across the memory bus.

First encounter with the memory wall: **we are not compute-bound, so making compute faster buys less than advertised.** 3x for one line is still an excellent trade.

Rung 2: BF16, and why FP16 needs a babysitter

Now we shrink the numbers themselves - 16 bits instead of 32.



IEEE normal-number limits; the 10^{-7} marker is illustrative

BF16 (brain floating point) keeps FP32's 8 exponent bits, so the *range* is identical and only precision is lost. **FP16** spends 5 bits on the exponent, so small gradients fall off the bottom of its range - which is the entire reason **gradient scalers** exist.

A short history, because it explains the confusion

- ▶ **FP16 came first** (Volta). Everyone trained in it, and everyone had to wrap their training loop in gradient scaling to stop small gradients underflowing. Extra state, extra complexity, extra bugs.
- ▶ **BF16 arrived with Ampere** and deleted the problem. Same range as FP32, so **no gradient scaler at all**.

Most tutorials you will find online were written during the FP16 era and are full of scaler machinery you do not need.

If your GPU supports BF16, use BF16 and ignore every gradient scaler tutorial. If it does not (a Colab T4, for instance), you are back in the FP16 world and you *do* need the scaler.

Mixed precision: what actually changes

torch.autocast wraps the forward pass and the loss ONLY - not backward, not the optimizer step

Cast to BF16

linear layers (matmuls)

attention QK^T and PV

convolutions

-> activations become BF16

Left in FP32

layer norm

softmax

the loss (cross-entropy)

-> parameters stay FP32

schematic; matmul-like ops are cast, reduction-heavy ops are not

```
with torch.autocast(device_type='cuda', dtype=torch.bfloat16):  
    logits, loss = model(x, y)
```

Unlike TF32, this is visible: your logits come back as BF16 while your parameters stay FP32. That is what “mixed” means.

Two rules and an honest result

Wrap the forward pass and the loss only. Never the backward pass, never the optimizer step. This is the guidance from the PyTorch team, and it is easy to get wrong because wrapping more *looks* more thorough.

Do not call `.bfloat16()` on your tensors by hand. Let autocast decide. Which operations it casts is not documented as clearly as you would like - matmul-like ops get cast, reduction-heavy ops such as layer norm and softmax mostly do not.

333 ms → 300 ms. Real, but small.

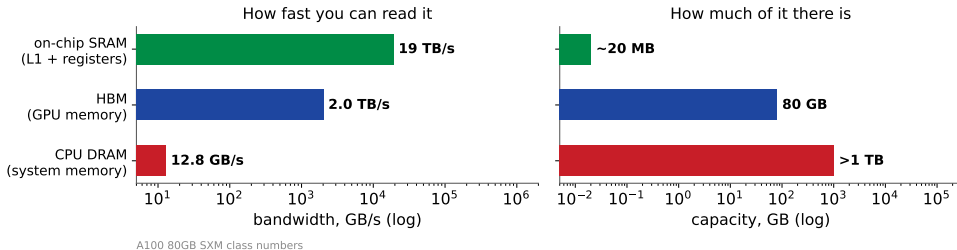
We have now spent everything precision alone can buy. The bottleneck has moved somewhere else - and to find it we have to look at how data physically travels inside the GPU.

Stop making round trips

Arithmetic is cheap. Moving bytes is expensive.

The memory wall, drawn to scale

The memory wall: the fast memory is tiny, the big memory is slow



On-chip **static random-access memory (SRAM)** is roughly ten times the bandwidth of **high-bandwidth memory (HBM)** and about four thousand times smaller. The CPU's **dynamic random-access memory (DRAM)** is enormous and hopelessly slow by comparison. Both axes are logarithmic.

Worked example: is that actually true?

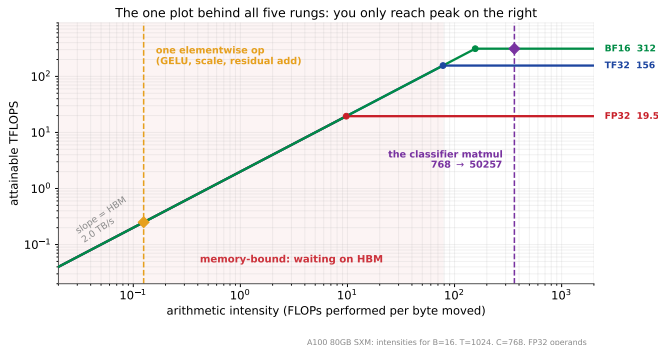
One elementwise operation on the MLP hidden activations, in FP32. Everything here is arithmetic you can do on paper.

Tensor shape	$16 \times 1024 \times 3072 = 50.3\text{M numbers}$
Bytes moved	read 201 MB + write 201 MB = 403 MB
Arithmetic	50.3M multiplies = 50.3 MFLOP
Time to move it, at 2.0 TB/s	201 μs
Time to compute it, at 19.5 TFLOPS	2.6 μs

78x more time moving the data than using it.

So “memory-bound” is not a figure of speech. For everything that is not a matrix multiply, the arithmetic is nearly free and the transfer is the whole cost - and the fix becomes obvious: **move it once, do everything, move it back.**

One plot behind the entire lecture



Arithmetic intensity = FLOPs performed per byte moved. Left of the bend you are memory-bound and the datasheet's peak is unreachable whatever precision you choose. Only matrix multiplies live on the right.

Reading the roofline, rung by rung

- ▶ **TF32 raised the ceiling** from 19.5 to 156 TFLOPS, and moved the bend from 9.8 to 78 FLOPs per byte. Our elementwise work sits at 0.125, far to the left of both. The ceiling was never the constraint - that is the 8x that became 3x.
- ▶ **BF16 moves us sideways as well as up.** Halving the bytes doubles the intensity, 0.125 to 0.25. That sideways step is why it helps at all.
- ▶ **Fusion is the move that changes intensity a lot.** Three round trips become one, so the same arithmetic is paid for with a third of the traffic.
- ▶ **FlashAttention is that same move**, applied to an operation the compiler is not able to restructure on its own.

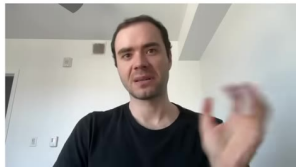
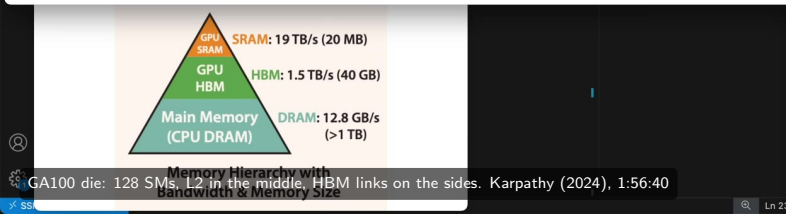
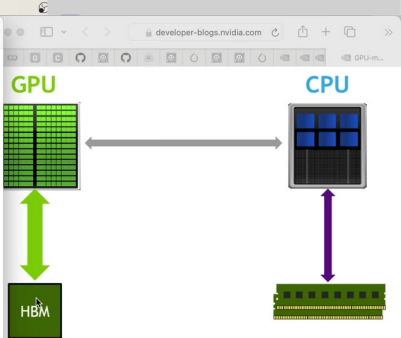
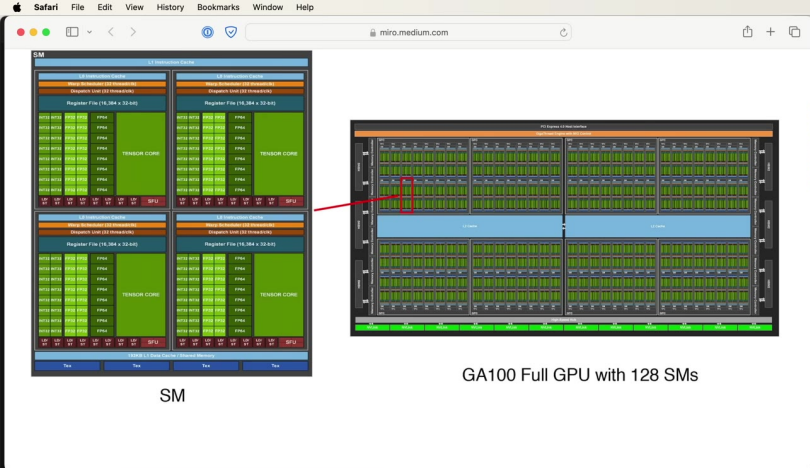
Four of our five rungs are attempts to move right on this plot or to stop caring about the ceiling. Only vocab padding is about something else entirely.

Where that memory physically sits

The GPU die is covered in **streaming multiprocessors** (SMs) - 108 active on an A100, out of 128 on the full chip. Each has tensor cores, ordinary arithmetic units, registers and L1 cache.

The large memory (HBM) is a *separate chip*. Every byte your kernel reads from it has to cross that gap.

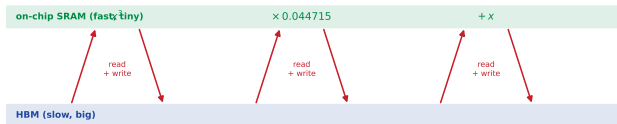
On the chip: lightning fast, almost no room.
Off the chip: all the room you want, and you wait.



One activation function, three pointless round trips

Take the **Gaussian Error Linear Unit (GELU)** and write it out by hand: x^{**3} , then $\times 0.044715$, then $+ x$. In eager PyTorch each of those is its own kernel.

Eager: one kernel per operation, a full round trip each time (3 reads + 3 writes)



Fused (torch.compile): one round trip for the whole chain (1 read + 1 write)



Each kernel streams the whole tensor out of HBM, does one cheap elementwise operation, and writes the whole tensor back. The arithmetic is trivial. **The travel is the cost.**

Rung 3: `torch.compile`

```
model = torch.compile(model)
```

Two separate wins, from one line:

1. **The Python interpreter leaves the forward pass.** Eager mode walks your code operation by operation with no idea what comes next. The compiler sees the whole graph at once and emits it as a single object.
2. **Kernel fusion.** Knowing the whole chain, it loads a chunk of the tensor into on-chip memory, performs *every* elementwise operation there, and writes back **once**.

300 ms → **130 ms**. A 2.3x speedup.

PyTorch's own summary: "speedup mainly comes from reducing Python overhead and GPU read/writes". Those are exactly the two wins above.

What it costs you

- ▶ **Compilation time up front.** The first iteration is slow, sometimes dramatically. This is a second reason to throw the first step out of your timing.
- ▶ **Debugging gets harder.** The graph you are stepping through is no longer the Python you wrote.

Karpathy's position, and it is a reasonable default: **use it unless you are actively debugging.**

Note where this sits in the ranking. TF32 is still the biggest single rung (3.0x against compile's 2.3x), and the two precision rungs together beat it as well. But TF32 and BF16 bought their speed by **throwing away mantissa bits**; `torch.compile` is the largest win that costs no accuracy at all. It only stopped us walking to HBM and back for no reason.

What the compiler cannot find

Two more wins, both of which you have to know about yourself.

Rung 4: FlashAttention

FlashAttention (Dao et al., 2022) is **also** a kernel fusion. So why did `torch.compile` not find it?

Because it is not a graph optimization. It requires an **algorithmic rewrite**: computing softmax incrementally so the $T \times T$ attention matrix is **never materialized** in HBM at all.

That trick - the **online softmax** - was published by Milakov and Gimelshein at NVIDIA in **2018**, four years before FlashAttention used it. They even predicted it would help. Nobody connected it to attention until Stanford did.

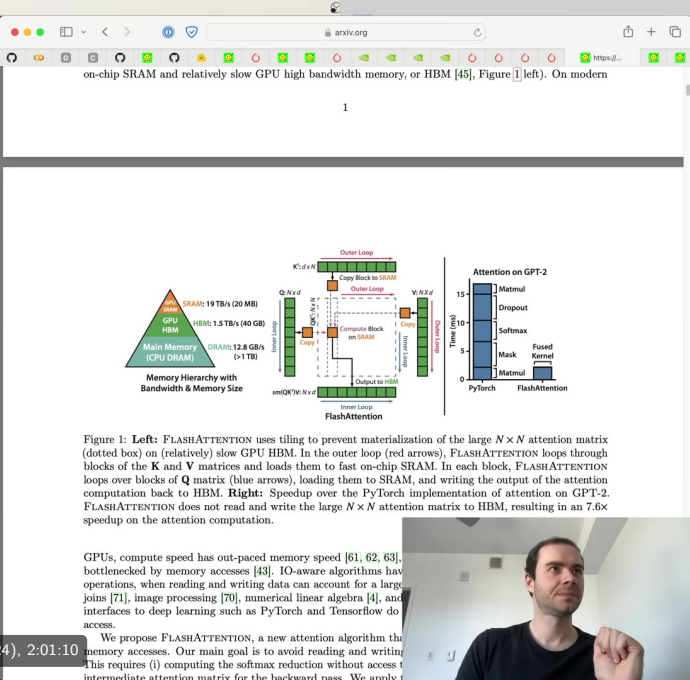
The tiling and the online-softmax recursion are derived properly in **deck 15 (FlashAttention)**. Today it is one rung on a ladder.

```
Safari File Edit View Help
train_gpt2.py
train_gpt2.py CausalSelfAttention > forward
9 class CausalSelfAttention(nn.Module):
25
26 def forward(self, x):
27     B, T, C = x.size() # batch size, sequence length, embedding dimension
28     # calculate query, key, values for all heads in batch and move head for
29     # nh is "number of heads", hs is "head size", and C (number of channels)
30     # e.g. in GPT-2 (124M), n_head=12, hs=64, so nh*hs=C=768 channels in the
31     qkv = self.c_attn(x)
32     q, k, v = qkv.split(self.n_embd, dim=2)
33     k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, n
34     q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, n
35     v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, n
36     # attention (materializes the large (T,T) matrix for all the queries and
37
38     att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1)))
39     att = att.masked_fill(self.bias[:, :, :T, T] == 0, float('-inf'))
40     att = F.softmax(att, dim=-1)
41     att = att @ v # (B, nh, T, T) x (B, nh, T, hs) -> (B, nh, T, hs)
42
43     y = y.transpose(1, 2).contiguous().view(B, T, C) # re-assemble all head
44     # output projection
45     y = self.c_proj(y)
46     return y
47
48 class MLP(nn.Module):
49
50 def __init__(self, config):
51     super().__init__()
52     self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd)
53     self.gelu = nn.GELU(approximate='tanh')
```

PROBLEMS DEBUG CONSOLE TERMINAL PORTS 1 JUPYTER

TERMINAL

```
step 40, loss: 6.258200645446777, dt: 129.62ms, tok/sec: 126401.80
step 41, loss: 6.005910396575928, dt: 129.59ms, tok/sec: 126429.71
step 42, loss: 6.147232532501221, dt: 129.94ms, tok/sec: 126091.94
step 43, loss: 6.036440849304199, dt: 129.46ms, tok/sec: 126554.51
step 44, loss: 5.985895156860352, dt: 129.77ms, tok/sec: 126249.46
step 45, loss: 6.204989910125732, dt: 129.73ms, tok/sec: 126296.57
step 46, loss: 6.325179576873779, dt: 129.78ms, tok/sec: 126246.91
step 47, loss: 6.194539546966553, dt: 129.64ms, tok/sec: 126384.13
step 48, loss: 6.135073661804100, dt: 130.13ms, tok/sec: 125900.20
```



GPUs, compute speed has out-paced memory speed [61, 62, 63], bottlenecked by memory accesses [43]. IO-aware algorithms have operations, when reading and writing data can account for a large joins [71], image processing [70], numerical linear algebra [4], and interfaces to deep learning such as PyTorch and Tensorflow do access.

We propose FLASHATTENTION, a new attention algorithm that memory accesses. Our main goal is to avoid reading and writing. This requires (i) computing the softmax reduction without access to intermediate attention matrix for the backward pass. We apply t



FlashAttention (Dao et al., 2022). Figure 1: Shown in Karpathy (2024), 2:01:10

Predict first: it does more arithmetic

FlashAttention performs **more** floating-point operations than the standard implementation. Count them and it loses.

So is it slower?

Predict first: it does more arithmetic

FlashAttention performs **more** floating-point operations than the standard implementation. Count them and it loses.

So is it slower?

130 ms → **96 ms**. About 27% faster.

It never writes the big attention matrix to slow memory, and that saving dwarfs the extra arithmetic.

The lesson to carry out of this room: **FLOPs are not the cost model. Memory access is.** A FLOP count can rank an algorithm as the worse one when it is in fact the faster one.

The change itself

Four lines of hand-written attention:

```
att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1)))
att = att.masked_fill(bias[:, :, :T, :T] == 0, float('-inf'))
att = F.softmax(att, dim=-1)
y = att @ v
```

become one:

```
y = F.scaled_dot_product_attention(q, k, v, is_causal=True)
```

You have to ask for it explicitly. PyTorch will not rewrite your four lines into this call, even though it recognizes the pattern well enough to provide the fused version.

Rung 5: nice numbers and ugly numbers

CUDA kernels chop work into **block tiles** that are powers of two - 16, 32, 64, 128. When your shapes divide evenly, every tile is full. When they do not, a second clean-up kernel runs for the remainder, and it is far less efficient.

So scan the model config for numbers that are not friendly:

Value	Verdict	Why
1024 (block size)	nice	2^{10}
768 (embedding dim)	nice	256×3
12 (layers, heads)	fine	divisible by 4
50257 (vocab)	ugly	29×1733 , odd, no factor of two
25 (heads, GPT-2 XL)	ugly	odd

Predict first: make the model bigger to make it faster

We pad the vocabulary from **50257** up to **50304**, which is $2^7 \times 393$ and divides by 128. That is 47 extra tokens, a bigger embedding table, a bigger classifier, and **strictly more arithmetic**.

Faster or slower?

Predict first: make the model bigger to make it faster

We pad the vocabulary from **50257** up to **50304**, which is $2^7 \times 393$ and divides by 128. That is 47 extra tokens, a bigger embedding table, a bigger classifier, and **strictly more arithmetic**.

Faster or slower?

96 ms $\rightarrow$ **93 ms**. About 3% faster.

That is **0.09% more arithmetic** traded for **3% less time**. And on PyTorch 2.3.1 or earlier the same change was worth roughly **30%**; the kernels have since got better at ragged shapes, which is itself worth knowing: *these numbers rot*.

Why padding the vocabulary is safe

A fair objection: we just invented 47 tokens that do not exist. Does that break anything?

- ▶ **At the embedding table:** the tokenizer only ever emits ids up to 50256, so the added rows are **never indexed**. They cost memory and nothing else.
- ▶ **At the classifier:** we now predict 47 extra logits. The optimizer drives them toward negative infinity, exactly as it already does for the thousands of real tokens that never appear in our data.

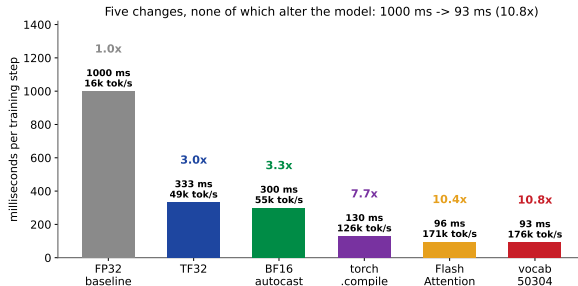
For context, 50257 is itself an awkward artifact: 50,000 byte-pair merges, plus 256 byte tokens, plus one `<|endoftext|>`. Nobody chose it for the hardware.

Functionally identical, measurably faster. This is the one optimization that feels like cheating and is not.

The whole ladder

Eleven times faster, and the model never changed.

Five rungs



A100 80GB SXM, GPT-2 124M, batch 16 x 1024 tokens (Karpathy, 2024)

Notice where the money was. The two precision rungs together bought 3.3x; `torch.compile` alone more than doubled that again. The last two rungs are small in percentage terms and are the ones that require you to actually understand the hardware.

Which rungs cost you accuracy?

Rung	Result	What you trade
TF32	approximate	13 mantissa bits, invisibly
BF16 autocast	approximate	16 mantissa bits on some ops
<code>torch.compile</code>	same computation	compile time, debuggability
FlashAttention	same computation	nothing
vocab padding	same computation	a little memory

Only the first two are genuine trades, and both are trades the field has decided are worth making. You lose a little numerical precision and you can **train for longer with the time you saved**, which more than pays it back.

“Same computation” means algebraically equivalent, not bit-identical - floating-point addition is not associative, so reordering work changes the last digits. Expect losses to match to about four decimal places, not exactly.

How do you know you did not break it?

Every rung here is a performance change, and performance changes are exactly where silent bugs hide. The discipline that catches them is dull and non-negotiable:

1. Fix the seed. Record the loss at a **fixed iteration** before the change.
2. Make **one** change.
3. Re-run and compare that same loss.

When FlashAttention went in, the loss at iteration 49 matched the previous run to about four decimal places - not bit-identical, because reordering floating-point work changes the last digits, but far closer than any genuine bug would leave it. When BF16 went in, it moved a little more, and that was expected.

A speedup that also moves your loss is a **bug** until you can name the approximation responsible. "It got faster" is not a test.

Reality check: you are probably not on an A100

Not every rung exists on every GPU			
	T4 (Turing, Colab free)	A100 (Ampere)	H100 (Hopper)
TF32	no	yes	yes
BF16 autocast	no	yes	yes
FP16 + gradient scaler	yes	yes	yes
torch.compile	yes	yes	yes
FlashAttention kernel	fallback	yes	yes
power-of-two shapes	yes	yes	yes

TF32 and BF16 arrived with Ampere; on Turing the FlashAttention kernel is unavailable and PyTorch uses its memory-efficient backend

TF32 and BF16 both arrived with **Ampere**. On a Colab T4 (Turing) neither exists, so the precision rung becomes **FP16 plus a gradient scaler** - back to the era the tutorials were written in. `torch.compile` and power-of-two shapes still work everywhere.

What actually transfers

The specific numbers in this deck belong to one GPU in 2024. The method does not.

1. **Measure first, and synchronize before you measure.** Most reported speedups are timing bugs.
2. **Find out what your hardware promises,** then find out what fraction you are getting. The gap is where the work is.
3. **Assume you are memory-bound until proven otherwise.** It explained four of our five rungs.
4. **FLOPs are not the cost model.** FlashAttention does more of them and wins.
5. **Give the kernels powers of two.** Free, and occasionally worth 30%.

All five changes together are about six lines of code. The hard part was never the typing - it was knowing which six.

Recap

- ▶ **1000 ms to 93 ms**, roughly **11x**, on an unchanged GPT-2 124M.
- ▶ **Precision** (TF32, then BF16) is the cheapest rung, and disappointed us relative to the datasheet because we are **memory-bound**.
- ▶ `torch.compile` was the biggest win that costs no accuracy (2.3x): it removes the Python interpreter from the forward pass and **fuses elementwise chains** so data stays on-chip.
- ▶ **FlashAttention** is the fusion a compiler cannot discover, because it needs an algorithmic rewrite (online softmax).
- ▶ **Nice numbers** matter, because CUDA tiles in powers of two.

Next: the other half of “let’s reproduce GPT-2” - **gradient accumulation**, AdamW with gradient clipping, the **warmup plus cosine** schedule, and **distributed data parallel** across eight GPUs. For the algorithm behind rung 4 see *deck 15 (FlashAttention)*; for these ideas at frontier scale, including **FP8** training, see *deck 17 (DeepSeek-V3)*.